

SOFTWARE TESTING



Pairwise Testing

Consider Situations:

A Web site must operate correctly with different browsers—

Internet Explorer 5.0, 5.5, and 6.0, Netscape 6.0, 6.1, and 7.0, Mozilla 1.1, and Opera 7;

using different plug-ins—RealPlayer, MediaPlayer, or none;

running on different client operating systems—Windows 95, 98, ME, NT, 2000, and XP;

receiving pages from different servers—IIS, Apache, and WebLogic;

running on different server operating systems—Windows NT, 2000, and Linux.

Combinations: $8 \times 3 \times 6 \times 3 \times 3 = 1296$

Worst



- Don't test at all. Simply give up because the number of input combinations, and thus the number of test cases, is just too great.
- Test all combinations [once], but delay the project so it misses its market window so that everyone quits from stress, or the company goes out of business.
- Choose one or two tests and hope for the best.
- Choose the tests that you have already run, perhaps as part of programmer-led testing. Incorporate them into a formal test plan and run them again.
- Choose the tests that are easy to create and run. Ignore whether they provide useful information about the quality of the product.
- Make a list of all the combinations and choose the first few.
- Make a list of all the combinations and choose a random subset.



- By magic, choose a specially selected, fairly small subset that finds a great many defects—more than you would expect from such a subset.

The answer is not to attempt to test all the combinations for all the values for all the variables but to test **all pairs** of variables.

If a system had four different input parameters and each one could take on one of three different values, the number of combinations is 3^4 which is 81. It is possible to cover all the pairwise input combinations in only nine tests.

If a system had thirteen different input parameters and each one could take on one of three different values, the number of combinations is 3^{13} which is 1,594,323. It is possible to cover all the pairwise input combinations in only fifteen tests.

If a system had twenty different input parameters and each one could take on one of ten different values, the number of combinations is 10^{20} . It is possible to cover all the pairwise input combinations in only 180 tests.

Why does pairwise testing work so well? I don't know.

There is no underlying "software physics" that requires it.

One hypothesis is that most defects are either single-mode defects (the function under test simply does not work and any test of that function would find the defect) or they are double-mode defects (it is the pairing of this function/module with that function/module that fails even though all other pairings perform properly).

Pairwise testing defines a minimal subset that guides us to test for all single-mode and double-mode defects.

The success of this technique on many projects, both documented and undocumented, is a great motivation for its use.

$L_4(2^3)$

Orthogonal

Array

	1	2	3
1	1	1	1
2	1	2	2
3	2	1	2
4	2	2	1

Maximum value = 2, 3, ..., N

Number of columns

$L_4(2^3)$

Number of rows

The following is an $L_{18}(3^5)$ orthogonal array. It has five columns, each containing a 1, 2, or 3. Examine columns 1 and 2 for the pair $\{1,1\}$. Does that pair exist in those two columns? Wait! Don't look at the array. From the definition of an orthogonal array, what is the answer? Yes, that pair exists along with every other pair of 1, 2, and 3. The pair $\{1,1\}$ is in row 1. Note that $\{1,1\}$ also appears in row 6. Returning to the original description of orthogonal arrays,

An orthogonal array is a two-dimensional array of numbers that has this interesting property—choose any two columns in the array. All the pairwise combinations of its values will occur in every column pair.

This definition is not totally complete. Not only will all the pair combinations occur in the array, but if any pair occurs multiple times, all pairs will occur that same number of times. This is because orthogonal arrays are "balanced." Examine columns 3 and 5—look for $\{3,2\}$. That combination appears in rows 6 and 17.

Orthogonal

Array

	1	2	3	4	5
1	1	1	1	1	1
2	1	2	3	3	1
3	1	3	2	3	2
4	1	2	2	1	3
5	1	3	1	2	3
6	1	1	3	2	2
7	2	2	2	2	2
8	2	3	1	1	2
9	2	1	3	1	3
10	2	3	3	2	1
11	2	1	2	3	1
12	2	2	1	3	3
13	3	3	3	3	3
14	3	1	2	2	3
15	3	2	1	2	1

1	6	3	1	1	3	2
1	7	3	2	3	1	2
1	8	3	3	2	1	1

1 1 1 1 1 1

Using Orthogonal Arrays

The process of using orthogonal arrays to select pairwise subsets for testing is:
Identify the variables.

- Determine the number of choices for each variable.
- Locate an orthogonal array which has a column for each variable and values within the columns that correspond to the choices for each variable.
- Map the test problem onto the orthogonal array.
- Construct the test cases.

Consider the first example in the introduction describing the software combinations a Web site must operate with.

Identify the variables.

The variables are Browser, Plug-in, Client operating system, Server, and Server operating system.

Determine the number of choices for each variable.

Browser - Internet Explorer 5.0, 5.5, and 6.0, Netscape 6.0, 6.1, and 7.0, Mozilla 1.1, and Opera 7 (8 choices).

Plug-in - None, RealPlayer, and MediaPlayer (3 choices).

Client operating system - Windows 95, 98, ME, NT, 2000, and XP (6 choices).

Server - IIS, Apache, and WebLogic (3 choices).

Server operating system - Windows NT, 2000, and Linux (3 choices).

Multiplying $8 \times 3 \times 6 \times 3 \times 3$ we find there are 1,296 combinations. For "complete" test coverage, each of these combinations should be tested.

Locate an orthogonal array that has a column for each variable and values within the columns that correspond to the choices of each variable.

What size array is needed? First, it must have five columns, one for each variable in this example. The first column must support eight different levels (1 through 8). The second column must support three levels (1 through 3). The third requires six levels. The fourth and the fifth each require three levels. The perfect size orthogonal array would be 816133 (one column of 1 through 8, one column of 1 through 6, and three columns of 1 through 3).

Unfortunately, one of this exact size does not exist. When this occurs, we simply pick the next larger array.

Map the test problem onto the orthogonal array.

The Browser choices will be mapped onto column 1 of the orthogonal array. Cells containing a 1 will represent IE 5.0; cells with a 2 will represent IE5.5; cells with a 3 will represent IE 6.0; etc. The mapping is:

1 ↔ IE 5.0

2 ↔ IE 5.5

3 ↔ IE 6.0

4 ↔ Netscape 6.0

5 ↔ Netscape 6.1

6 ↔ Netscape 7.0

7 ↔ Mozilla 1.1

8 ↔ Opera 7

If the perfect size array does not exist, choose one that is slightly bigger and apply these two rules to deal with the "excess." The first rule deals with extra columns. If the orthogonal array chosen has more columns than needed for a particular test scenario, simply delete them. The array will remain orthogonal.

The second rule deals with extra values for a variable. In the current example, column 3 runs from 1 to 8 but only 1 through 6 is needed. It is tempting to delete the rows that contain these cells but DON'T. The "orthogonality" may be lost.

If you delete a row, you lose that test case. Instead of deleting them, simply convert the extra cells to valid values. Some automated tools randomly choose from the set of valid values for each cell while others choose one valid value and use it in every cell within a column.

Allpairs Algorithm

Using orthogonal arrays is one way to identify all the pairs. A second way is to use an algorithm that generates the pairs directly without resorting to an "external" device like an orthogonal array.

All-Pairs Testing is a method where the test set is designed to ensure that for every combination of any two input parameters, all possible pairings of their values appear at least once.

James Bach presents an algorithm to generate all pairs in *Lessons Learned in Software Testing*. In addition, he provides a program called "Allpairs" that will generate the all pairs combinations. It is available at <http://www.satisfice.com>. Click on "Test Methodology" and look for Allpairs.

Final Comments

In some situations, constraints exist between certain choices of some of the variables. For example, Microsoft's IIS and Apple's MacOS are not compatible. It is certain that the pairwise techniques will choose that combination for test. (Remember, it does select **all** the pairs.) When creating pairwise subsets by hand, honoring these various constraints can be difficult. Both the rdExpert and AETG tools have this ability. You define the constraints and the tool selects pairs meeting those constraints

Given the two approaches to pairwise testing, orthogonal arrays and the Allpairs algorithm, which is more effective? One expert, who favors orthogonal arrays, believes that the coverage provided by Allpairs is substantially inferior. He notes that the uniform distribution of test points in the domain offers some coverage against faults that are more complex than double-mode faults

Another expert, who favors the Allpairs approach, notes that Allpairs does, in fact, test all the pairs, which is the goal. He claims there is no evidence that the orthogonal array approach detects more defects.

What both experts acknowledge is that no documented studies exist comparing the efficacy of one approach over the other.

What is Pairwise Testing?

- Also referred to as “all-pair/two-way testing”.
- It is a special case of all-combination testing of a system with n input variables.
 - > Let us consider n input variables denoted by $\{v_1, v_2, \dots, v_i, \dots, v_n\}$.
Eg. Browser, OS
 - > For each variable v_i , $1 \leq i \leq n$, we choose k values of interest.
Eg. For Browser: Mozilla and Internet Explorer
For OS: Linux and Windows
- An all-combination testing means considering all the k^n different test vectors. (where k are the values of interest for each variable and n is the total number of variables)

In this case: 2^2 different test vectors

What is Pairwise Testing?

- On the other hand, pairwise testing means covering all possible discrete combinations of input parameters by using a minimum set of test cases.

Example:

- For the three given variables X, Y, and Z, their value sets are as follows:
 $D(X) = \{\text{True}, \text{False}\}$, $D(Y) = \{0, 5\}$, and $D(Z) = \{Q, R\}$.

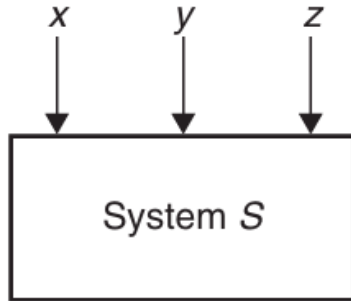


Figure 9.7 System S with three input variables.

All-Combination Test Cases: $2 \times 2 \times 2 = 2^3 = (k^n) = 8$

Test Case ID	Input (X)	Input (Y)	Input (Z)
1	True	0	Q
2	True	0	R
3	True	5	Q
4	True	5	R
5	False	0	Q
6	False	0	R
7	False	5	Q
8	False	5	R

Pairwise Test Cases: 4

Test Case ID	Input (X)	Input (Y)	Input (Z)
1	True	0	Q
4	True	5	R
6	False	0	R
7	False	5	Q

Test Case ID	Input (X)	Input (Y)	Input (Z)
1	True	0	Q
2	True	5	R
3	False	0	R
4	False	5	Q

What is Pairwise Testing?

- Pairwise testing would detect approximately 70% of the defects, whereas six-way testing would detect 100% of defects for browser and server applications.
- Pairwise testing is helpful when testing complex systems that have multiple input parameters and multiple possible values for each parameter.
- If a system has only a few parameters and a small number of possible values for each parameter, pairwise testing may be unnecessary.

Advantages:

- > Help identify bugs which would be difficult to otherwise.
- > Reduce number of test cases.

Disadvantages:

- > Time Consuming
- > May miss defects if input variables not chosen accurately.

Orthogonal Testing



What is Orthogonal testing?

- Statistical approach for testing pair-wise interactions that efficiently explores the interaction between different input parameters

Orthogonal vs pairwise?

- Orthogonal arrays ensure **balanced distribution** of data across columns and rows.

TABLE 9.6 Various Values That Need to Be Tested in Combinations

Variables	Values
Browser	Netscape, Internet Explorer (IE), Mozilla
Plug-in	Realplayer, Mediaplayer
OS	Windows, Linux, Macintosh
Connection	LAN, PPP, ISDN

TABLE 9.8 $L_9(3^4)$ Orthogonal Array after Mapping Factors

Test Case ID	Browser	Plug-in	OS	Connection
TC ₁	Netscape	Realplayer	Windows	LAN
TC ₂	Netscape	2	Linux	PPP
TC ₃	Netscape	Mediaplayer	Macintosh	ISDN
TC ₄	IE	Realplayer	Linux	ISDN
TC ₅	IE	2	Macintosh	LAN
TC ₆	IE	Mediaplayer	Windows	PPP
TC ₇	Mozilla	Realplayer	Macintosh	PPP
TC ₈	Mozilla	2	Windows	ISDN
TC ₉	Mozilla	Mediaplayer	Linux	LAN

$$L_{\text{Runs}} (\text{Levels}^{\text{Factors}})$$

- Runs (N): Number of rows, determines test cases.
- Factors (K): Number of columns, limits variables.
- Levels (V): Max values per factor.

No of Factors: 4

Maximum Levels: 3

Orthogonal technique:

- Ensures testing all possible pairs of selected variables.
- Produces fewer test cases compared to testing all combinations.
- Creates a test suite with an equal spread of all pairs of variables.
- Can be automated for efficient execution.

Steps of a technique to generate orthogonal arrays

1) Identify the maximum number of independent input variables with which a system will be tested. This will map to the factors of the array—each input variable maps to a different factor.

$$L_{\text{Runs}} (\text{Levels}^{\text{Factors}})$$

Variables	Values
Browser	Netscape, Internet Explorer (IE), Mozilla
Plug-in	Realplayer, Mediaplayer
OS	Windows, Linux, Macintosh
Connection	LAN, PPP, ISDN

2) Identify the maximum number of values that each independent variable will take. This will map to the levels of the array.

$$L_{\text{Runs}} (\text{Levels}^{\text{Factors}})$$

Variables	Values
Browser	Netscape, Internet Explorer (IE), Mozilla
Plug-in	Realplayer, Mediaplayer
OS	Windows, Linux, Macintosh
Connection	LAN, PPP, ISDN

3) Find a suitable orthogonal array with the smallest number of runs $L_{\text{Runs}}(X^Y)$, where X is the number of levels and Y is the number of factors

A suitable array is one that has *at least* as many factors as needed from step 1 and has *at least* as many levels for each of those factors as identified in step 2.

TABLE 9.7 $L_9(3^4)$ Orthogonal Array

Runs	Factors			
	1	2	3	4
1	1	1	1	1
2	1	2	2	2
3	1	3	3	3
4	2	1	2	3
5	2	2	3	1
6	2	3	1	2
7	3	1	3	2
8	3	2	1	3
9	3	3	2	1

4) Map the variables to the factors and values of each variable to the levels on the array.

TABLE 9.8 $L_9(3^4)$ Orthogonal Array after Mapping Factors

Test Case ID	Browser	Plug-in	OS	Connection
TC ₁	Netscape	Realplayer	Windows	LAN
TC ₂	Netscape	2	Linux	PPP
TC ₃	Netscape	Medioplayer	Macintosh	ISDN
TC ₄	IE	Realplayer	Linux	ISDN
TC ₅	IE	2	Macintosh	LAN
TC ₆	IE	Medioplayer	Windows	PPP
TC ₇	Mozilla	Realplayer	Macintosh	PPP
TC ₈	Mozilla	2	Windows	ISDN
TC ₉	Mozilla	Medioplayer	Linux	LAN

5) Check for any “left-over” levels in the array that have not been mapped. Choose valid values for those left-over levels.

1) Arbitrary

2) Cyclic rotation

TABLE 9.9 Generated Test Cases after Mapping Left-Over Levels

Test Case ID	Browser	Plug-in	OS	Connection
TC ₁	Netscape	Realplayer	Windows	LAN
TC ₂	Netscape	Realplayer	Linux	PPP
TC ₃	Netscape	Medioplayer	Macintosh	ISDN
TC ₄	IE	Realplayer	Linux	ISDN
TC ₅	IE	Medioplayer	Macintosh	LAN
TC ₆	IE	Medioplayer	Windows	PPP
TC ₇	Mozilla	Realplayer	Macintosh	PPP
TC ₈	Mozilla	Realplayer	Windows	ISDN
TC ₉	Mozilla	Medioplayer	Linux	LAN

6) Transcribe the runs into test cases

We generate nine test cases taking the test case values from each run.
Now let us examine the result:

- Each Browser is tested with every Plug-in, with every OS, and with every Connection.
- Each Plug-in is tested with every Browser, with every OS, and with every Connection.
- Each OS is tested with every Browser, with every Plug-in, and with every Connection.
- Each Connection is tested with every Browser, with every Plug-in, and with every OS

Total number of test cases using All-Combination Testing: $3 \times 3 \times 3 \times 2 = 54$

Total number of test cases using Pairwise Testing = 9

TABLE 9.9 Generated Test Cases after Mapping Left-Over Levels

Test Case ID	Browser	Plug-in	OS	Connection
TC ₁	Netscape	Realplayer	Windows	LAN
TC ₂	Netscape	Realplayer	Linux	PPP
TC ₃	Netscape	Medioplayer	Macintosh	ISDN
TC ₄	IE	Realplayer	Linux	ISDN
TC ₅	IE	Medioplayer	Macintosh	LAN
TC ₆	IE	Medioplayer	Windows	PPP
TC ₇	Mozilla	Realplayer	Macintosh	PPP
TC ₈	Mozilla	Realplayer	Windows	ISDN
TC ₉	Mozilla	Medioplayer	Linux	LAN

Values

Netscape, Internet Explorer (IE), Mozilla → 3

Realplayer, Medioplayer → 2

Windows, Linux, Macintosh → 3

LAN, PPP, ISDN → 3

Thank You